

File Handling in C

- File handling in C is the process in which we create, open, read, write, and close operations on a file.
- C language provides different functions such as fopen(), fwrite(), fread(), fseek(), fprintf(), etc. to perform input, output, and many different C file operations in our program.

Why do we need File Handling in C?

- So far the operations using the C program are done on a prompt/terminal which is not stored anywhere.
- The output is deleted when the program is closed. But in the software industry, most programs are written to store the information fetched from the program.
- The use of file handling is exactly what the situation calls for.

Features of using file

- ◆ **Reusability:** The data stored in the file can be accessed, updated, and deleted anywhere and anytime providing high reusability.
- ◆ **Portability:** Without losing any data, files can be transferred to another in the computer system. The risk of flawed coding is minimized with this feature.
- ◆ **Efficient:** A large amount of input may be required for some programs. File handling allows you to easily access a part of a file using few instructions which saves a lot of time and reduces the chance of errors.
- ◆ **Storage Capacity:** Files allow you to store a large amount of data without having to worry about storing everything simultaneously in a program.

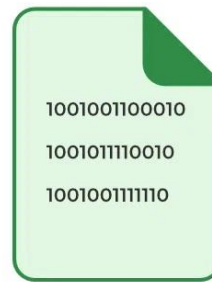
Types of Files in C

1. Text Files
2. Binary Files

Types of Files in C



file.txt



file.bin

1. Text Files

A text file contains data in the **form of ASCII characters** and is generally used to store a stream of characters.

- Each line in a text file ends with a new line character ('\n').
- It can be read or written by any text editor.
- They are generally stored with **.txt** file extension.
- Text files can also be used to store the source code.

2. Binary Files

A binary file contains data in **binary form (i.e. 0's and 1's)** instead of ASCII characters. They contain data that is stored in a similar manner to how it is stored in the main memory.

- The binary files can be created only from within a program and their contents can only be read by a program.
- More secure as they are not easily readable.
- They are generally stored with **.bin** file extension.

Functions for file handling

No.	Function	Description
1	fopen()	opens new or existing file
2	fprintf()	write data into the file
3	fscanf()	reads data from the file
4	fputc()	writes a character into the file
5	fgetc()	reads a character from file
6	fclose()	closes the file
7	fseek()	sets the file pointer to given position
8	fputw()	writes an integer to file
9	fgetw()	reads an integer from file
10	ftell()	returns current position
11	rewind()	sets the file pointer to the beginning of the file

Opening File: fopen()

We must open a file before it can be read, write, or update. The fopen() function is used to open a file.

Syntax:

```
FILE *fopen( const char * filename, const char * mode );
```

The fopen() function accepts two parameters:

- ◆ The file name (string). If the file is stored at some specific location, then we must mention the path at which the file is stored. For example, a file name can be like "c://some_folder/some_file.ext".
- ◆ The mode in which the file is to be opened. It is a string.

Closing File: fclose()

The fclose() function is used to close a file. The file must be closed after performing all the operations on it.

Syntax:

```
int fclose( FILE *fp );
```

Writing File : fprintf() function

The fprintf() function is used to write set of characters into file. It sends formatted output to a stream.

Syntax:

```
int fprintf(FILE *stream, const char *format [, argument, ...])
```

Reading File : fscanf() function

The fscanf() function is used to read set of characters from file. It reads a word from the file and returns EOF at the end of file.

Syntax:

```
int fscanf(FILE *stream, const char *format [, argument, ...])
```